Unit-5: Coding

Programming Languages:

- Set of commands, instructions, and other syntax used to create computer programs
- Allows programmers to communicate with computers and develop software
- Can be classified in many ways, but the most common is by **programming model (paradigm)**
    - **Imperative programming languages:**
        - Programs are written as a sequence of **computational steps** (instructions, statements, or commands) that the computer executes **step by step**.
        - Focuses on **how to compute**, not what to compute.
        - Programs are divided into smaller, reusable parts called procedures or subprograms
        - **E.g. FORTRAN, PASCAL**
    - **Functional programming languages:**
        - programming paradigm that treats computation as the evaluation of mathematical functions
        - Focuses on **what to compute**, not how to compute.
        - Data is immutable, meaning once created, it cannot be modified.
        - The function's output depends only on input, and it does not modify anything else.
        - Supports **recursion** instead of loops for repetition.
        - **E.g.** Lisp (List Processor), Miranda, SML (Standard ML Language)
    - **Object-oriented programming languages:**
        - Organizes programs into objects (real-world entities) and classes (blueprints).
        - **Supports encapsulation:** Binding data and methods together.
        - **Provides abstraction:** Hides details, shows only essentials.
        - **Enables inheritance:** Reusing and extending existing classes.
        - **Allows polymorphism:** Same function/method behaves differently in different contexts.
        - **E.**g. Java, C++, Python, C# etc.
        - **Uses:**
            - **Web Development:** Frameworks like Django & Laravel
            - **Large Software Applications:** Desktop software (e.g., Adobe Photoshop) & Game development (Unity C#)

- **Logical programming languages:**
  - Based on **formal logic** (facts and rules) to solve problems.
  - Programs are written as a set of **statements and relationships**.
  - Uses **inference and reasoning** instead of step-by-step instructions.
  - Most popular logical programming language is **Prolog.**
  - **Uses:**
    - **Expert Systems & AI:** Used in rule-based applications like medical diagnosis engines and chatbots.

Programming Development Tools:

- Tools that help developers create software efficiently, including:
  - **IDE (Integrated Development Environment):** Software to write, compile, and debug code in one place (e.g., Visual Studio, PyCharm, NetBeans).
  - **Frameworks:** Predefined code libraries that simplify application development (e.g., Diango (Python), Laravel (PHP)).
  - **Cloud Tools:** Online platforms to build, deploy, and manage applications (e.g., AWS, Google Cloud, Microsoft Azure).
  - **Version Control Tools:** Manage code versions and track changes (e.g., Git).

Selecting Languages and Tools:

- Choosing the **right language and development tools** to efficiently build a software project.
- Factors to consider:
  - **Project requirements:** Type of application, complexity, and platform (web, mobile, desktop).
  - **Performance needs:** Speed, memory usage, and scalability.
  - **Team expertise:** Skills and experience of developers.
  - **Stakeholder requirements:** Sometimes, the choice depends on client or stakeholder preferences.
  - **Tool availability and ease of use:** Prefer tools that are accessible and simple to handle.

Good Programming Practices:

- **Before writing software:**
  - **Build a good team:** Form a skilled and collaborative team
  - **Work with professionals:** Involve experienced developers and experts
  - **Understand software:** Clearly analyze requirements and objectives
  - **Select a development process:** Choose an appropriate development methodology
- **While writing software:**
  - **Avoid quick coding:** Do not rush; write carefully
  - **Follow coding standards:** Stick to consistent coding conventions
  - **Simplify coding:** Keep code clear and easy to understand
  - **Reuse existing code:** Use proven code modules to save time.
- **After writing software:**
  - **Review code:** Check for errors and improve code quality.
  - **Test code:** Verify that the software works correctly
  - **Maintain code:** Update and fix software as needed
  - **Create a user manual:** Provide instructions for users

Coding Standards:

- Set of rules or guidelines that programmers follow to write clean, consistent, and readable code
- **Why Are They Important?**
  - **Readability:** Makes code easy for other developers (and your future self!) to understand quickly.
  - **Maintainability:** Fixing bugs or adding new features is much easier in clean, well-organized code.
  - **Team Collaboration:** Ensures everyone on a team writes code in the same style, making it seem like one person wrote it.
  - **Reduces Errors:** Consistent structure helps spot mistakes more easily.
  - **Professionalism:** Reflects discipline and best practices in software development.
- **Common Coding Standards (With Simple Examples):**
  - **Naming Conventions:**
    - **Use meaningful and descriptive names.**
      - **Bad:** int p; //what does 'p' mean?
      - **Good:** int profit;
    - **Variables & Functions:** Use **camelCase** (e.g., userName, calculateProfit()).

- **Constants:** Use **UPPER_SNAKE_CASE** (e.g., const MAX_LOGIN_ATTEMPTS = 5;).
- **Classes:** Use **PascalCase** (e.g., class UserAccount { }).

- **Indentation and Spacing:**
  - **Be consistent:** Use either tabs or spaces (commonly 2 or 4 spaces) throughout the project.
  - **Use whitespace to improve readability**.
    - **Bad:** if(condition){result=value;}
    - **Good:**

      ```
      if (condition) {
          result = value;
      }
      ```

- **Error Handling:**
  - **Handle errors gracefully**, don't let the program crash unexpectedly.
  - **Use meaningful error messages** that help with debugging.
- **Avoid "Magic Numbers":**
  - **Replace unexplained numbers or strings with named constants.**
    - **Bad:** if (userAge > 18) { ... } // What is 18?
    - **Good:** const LEGAL_AGE = 18; if (userAge > LEGAL_AGE) { ... }
- **Comments:**
  - **Use comments for complex logic** or business rules.
  - Explain **why you did something, not what you did** (the **code should be self-explanatory for the "what"**).
  - E.g.,

    ```
    // Using quick sort for performance with large datasets
    quickSort(userList);
    ```

- **Code Structure:**
  - **Keep functions small**, each doing one task.
  - **Limit line length (e.g., 80-120 characters)** to avoid horizontal scrolling.
  - **Organize files logically** (e.g., separate files for different classes, components).